LeetCode 509: Fibonacci Number

1. Problem Title & Link

Title: LeetCode 509: Fibonacci Number

Link: https://leetcode.com/problems/fibonacci-number/

2. Problem Statement (Short Summary)

The Fibonacci sequence is defined as:

F(0) = 0

F(1) = 1

For all values greater than 1:

F(n) = F(n-1) + F(n-2)

Given an integer n, return the value of F(n).

3. Examples (Input → Output)

Example 1

Input:

n = 2

Output:

1

Explanation:

F(2) = F(1) + F(0)

= 1 + 0

= 1

Example 2

Input:

n = 3

Output:

2

Explanation:

F(3) = F(2) + F(1)

= 1 + 1

= 2

Example 3

Input:

n = 4

Output:

3

Explanation:

F(4) = F(3) + F(2)

= 2 + 1

= 3

Example 4

Input:

n = 6

Output:

8

Sequence:

0 1 1 2 3 5 8

4. Constraints

0 <= n <= 30

Important:

F(0) = 0

F(1) = 1

These are the base cases.

5. Core Concept (Pattern / Topic)

Dynamic Programming

Secondary Topics:

- Recursion
- Memoization
- Fibonacci Pattern

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

The problem directly gives:

F(n)=F(n-1)+F(n-2)

So we can write recursion.

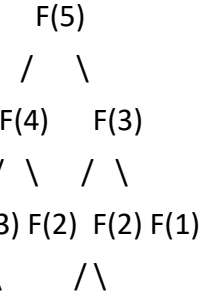
Example:

F(5)

= F(4) + F(3)

= (F(3)+F(2)) + (F(2)+F(1))

Recursive Tree



Notice:

F(3)

F(2)

are computed multiple times.

This creates unnecessary work.

Better Thinking (Dynamic Programming)

Store previously computed answers.

Instead of recomputing:

F(3)

use the already stored value.

This reduces complexity dramatically.

Best Observation

To calculate:

F(5)

we only need:

F(4)

F(3)

Similarly:

F(6)

needs:

F(5)

F(4)

Therefore, only the previous two values are required.

No array is necessary.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Fibonacci Sequence:
Index : 0 1 2 3 4 5 6 7

Value : 0 1 1 2 3 5 8 13

Building the sequence:
0 + 1 = 1

1 + 1 = 2

1 + 2 = 3

2 + 3 = 5

3 + 5 = 8

Visualization:
prev2    prev1

  2   +   3

       ↓

       5
Move forward:
prev2 = 3

prev1 = 5
Continue.
```

8. Pseudocode (Language Independent)

```
if n == 0
```

```
    return 0

if n == 1
    return 1

prev2 = 0
prev1 = 1

for i from 2 to n

    current = prev1 + prev2

    prev2 = prev1

    prev1 = current

return prev1
```

9. Code Implementation

Python

```
class Solution:
    def fib(self, n: int) -> int:

        if n == 0:
            return 0

        if n == 1:
            return 1

        prev2 = 0
        prev1 = 1

        for i in range(2, n + 1):

            current = prev1 + prev2

            prev2 = prev1
            prev1 = current

        return prev1
```

Java

```
class Solution {

    public int fib(int n) {

        if(n == 0)
            return 0;

        if(n == 1)
            return 1;

        int prev2 = 0;
        int prev1 = 1;

        for(int i = 2; i <= n; i++) {

            int current = prev1 + prev2;

            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

10. Time & Space Complexity

Iterative DP Solution

Time Complexity: O(n)

Space Complexity: O(1)

Reason:

One loop

Three variables

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting base cases.

Wrong:

for i in range(2, n+1)

without handling:

n = 0

n = 1

Mistake 2

Starting loop from wrong position.

Correct:

i = 2

because:

F(0)

F(1)

already known.

Mistake 3

Updating variables incorrectly.

Wrong:

prev2 = prev1

prev1 = prev1 + prev2

Uses modified values.

Edge Cases

Input	Output
0	0
1	1
2	1
3	2
4	3
5	5

12. Detailed Dry Run (Step-by-Step Table)

Input:

n = 6

Initial:

prev2 = 0

prev1 = 1

i	current	prev2	prev1
2	1	1	1

3	2	1	2
4	3	2	3
5	5	3	5
6	8	5	8

Return:

8

DP Array Visualization

dp[0] = 0

dp[1] = 1

dp[2] = 1

dp[3] = 2

dp[4] = 3

dp[5] = 5

dp[6] = 8

13. Common Use Cases (Real-Life / Interview)

Population Growth Models

Rabbit population

Biological growth

Financial Modeling

Compound growth patterns

Dynamic Programming Introduction

Fibonacci is often the first DP problem students learn.

Algorithm Design

Helps understand:

Recursion

Memoization

Tabulation

Space Optimization

14. Common Traps (Important!)

Trap 1

Using pure recursion.

fib(n-1) + fib(n-2)

Looks simple but becomes very slow.

Trap 2

Not identifying overlapping subproblems.

fib(3)

fib(2)

are computed repeatedly.

Trap 3

Using an array when only two values are needed.

Trap 4

Confusing Fibonacci indexing.

Remember:

F(0) = 0

F(1) = 1

Not:

F(1) = 0

F(2) = 1

15. Builds To (Related LeetCode Problems)

Dynamic Programming Roadmap:

LC 509 - Fibonacci Number

↓

LC 70 - Climbing Stairs

↓

LC 746 - Min Cost Climbing Stairs

↓

LC 198 - House Robber

↓

LC 213 - House Robber II

↓

LC 322 - Coin Change

Fibonacci is the foundation of many DP problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Pure Recursion	$O(2^n)$	O(n)	Poor
Memoization	O(n)	O(n)	Good
DP Array	O(n)	O(n)	Good
Iterative DP	O(n)	O(1)	Best
Matrix Exponentiation	O(log n)	O(1)	Advanced

Approach 1: Recursion

```
def fib(n):

  if n <= 1:
    return n

  return fib(n-1) + fib(n-2)
```

Simple but inefficient.

Approach 2: Memoization

Store previously calculated results.

Avoid repeated calculations.

Approach 3: Iterative DP (Chosen)

Track only:

Previous Fibonacci

Current Fibonacci

Most commonly expected in interviews.

17. Why This Solution Works (Short Intuition)

Each Fibonacci number depends only on the previous two Fibonacci numbers:

$F(n)=F(n-1)+F(n-2)$

By maintaining only these two values and updating them iteratively, we generate the sequence efficiently while using constant extra space.

18. Variations / Follow-Up Questions

Follow-Up 1

Print the first N Fibonacci numbers.

Example:

N = 7

Output:

0 1 1 2 3 5 8

Follow-Up 2

Find the sum of the first N Fibonacci numbers.

Example:

0+1+1+2+3+5

Follow-Up 3

Find the Nth Tribonacci Number.

Recurrence:

T(n) = T(n-1) + T(n-2) + T(n-3)

Related to Introduction to Algorithms style DP extensions.

Follow-Up 4

Find Fibonacci using matrix exponentiation in O(log n).

Frequently asked in advanced interviews.

Interview Takeaway

Fibonacci is the most important introductory Dynamic Programming problem.

The key learning progression is:

Recursion

↓

Memoization

↓

Tabulation

↓

Space Optimization

Whenever you see:

Current Answer

=

Combination of Previous Answers

think:

Dynamic Programming